

Execution-Guided Detection of Semantic Code Perturbations in AI-Generated Code

Aman Mukati
Independent Researcher
amanmukati2002@gmail.com

Abstract—Static analysis and embedding-based methods fail to detect semantic perturbations in AI-generated code—changes that preserve syntactic structure while altering program behavior. Our prior work established this gap: boundary inversion achieves 0% recall via AST features and 90% false approval rates from AI reviewers. This paper proposes execution-guided detection: a framework that runs code in a sandboxed environment, captures execution traces across diverse inputs, and classifies code as clean or perturbed based on behavioral differences. Using 550 AI-generated Python samples (100 clean, 450 perturbed across five strategies), we demonstrate that pairwise trace comparison achieves 87.5% recall on boundary inversion—a 87.5 percentage point improvement over static analysis. Import-aware features improve import aliasing detection from 13.3% to 58.2% recall. A hybrid detector combining execution traces with AST features achieves 0.904 accuracy and 0.942 F1 score. Wilcoxon signed-rank test confirms statistical significance ($p = 0.0074$). Our C-based sandbox executor and property-based test generator are publicly available.

Index Terms—adversarial code, execution-guided detection, semantic perturbations, sandbox execution, dynamic analysis, AI security

I. INTRODUCTION

AI code generation systems are increasingly deployed in production, with tools like GitHub Copilot, Codex, and Gemini generating code at scale. Yet the security implications of adversarially perturbed AI-generated code remain critically underexplored. What if an attacker could systematically induce vulnerabilities that pass human review and automated static analysis?

Our prior work established a **three-tier detectability hierarchy**:

- **Tier 1 (Trivially detected)**: Structural perturbations with 100% recall via AST features
- **Tier 2 (Feature-specific)**: Import aliasing with 7–17% recall
- **Tier 3 (Invisible)**: Boundary inversion with 0% recall

This paper proposes a fundamentally different approach: **execution-guided detection**. Instead of analyzing code structure or embeddings, we observe what code *does* when executed with diverse inputs.

Key insight: Semantic perturbations preserve syntactic structure but change program behavior. Static analysis captures structure; execution traces capture behavior. Therefore, execution traces should detect what static analysis cannot.

A. Contributions

Our contributions are:

- 1) A novel execution-guided detection framework for adversarial code perturbations
- 2) A C-based sandbox executor with timeout and resource limits
- 3) A property-based test input generator for Python functions
- 4) Empirical validation: 87.5% recall on boundary inversion (vs 0% static)
- 5) Import-aware features improving import aliasing detection to 58.2%
- 6) Hybrid detector achieving 0.904 accuracy and 0.942 F1

II. BACKGROUND AND RELATED WORK

A. AI Code Generation and Security

Large language models have transformed code generation. GitHub Copilot, Codex, and Gemini generate production code at scale. However, security analysis has focused primarily on accidental vulnerabilities.

Pearce et al. [3] studied security bugs in Copilot-generated code, finding that approximately 40% contain vulnerabilities. Chen et al. [4] evaluated Codex on programming tasks. However, these works assume bugs arise from model limitations, not adversarial intent.

B. Adversarial Code Perturbations

Our prior work [1], [2] systematically studied adversarial perturbations in AI-generated code. We identified five perturbation strategies:

TABLE I
PERTURBATION STRATEGIES STUDIED

Strategy	Type	Example
Comment planting	Structural	# dangerous_call()
Dead code insertion	Structural	if False: bypass()
Variable shadowing	Structural	result = copy(result)
Import aliasing	Semantic	from md5 import sha256
Boundary inversion	Semantic	< → <=

C. Static and Embedding-Based Detection

Our prior work established that AST features achieve 0% recall on boundary inversion and 7% on import aliasing. Mean-pooled CodeBERT [5] improves with scale but still fails on

semantic perturbations. Token-level representations provide marginal improvement but still 0% on boundary inversion.

D. Execution-Based Analysis

Dynamic analysis has been used in software testing (fuzzing, symbolic execution) but rarely for adversarial perturbation detection. Fuzzing [4] generates inputs to find crashes; our work uses execution traces to detect intentional perturbations.

III. METHODOLOGY

A. Threat Model

An adversary modifies AI-generated code after generation but before deployment. The modification:

- Preserves syntactic plausibility (passes human review)
- Evades static analysis tools
- Alters program behavior in a harmful way

B. System Architecture

Our system consists of five components:

- 1) **Property Generator:** Generates diverse test inputs based on function type hints
- 2) **Sandbox Executor (C):** Executes code in isolated subprocess with timeout
- 3) **Trace Extractor:** Captures outputs, exit codes, execution times
- 4) **Trace Comparator:** Computes differences between execution traces
- 5) **Detector:** Classifies code as clean or perturbed

C. Property-Based Test Input Generation

We generate test inputs based on function type hints. For integers, boundary values ($-1, 0, 1, 10^6$) are selected with 30% probability. Strings vary in length from 0 to 50 characters. Lists contain 0 to 10 elements.

Why boundaries matter: Boundary inversion flips comparison operators. Test inputs near the boundary are most likely to expose behavioral differences. Our generator explicitly biases toward boundary values.

Input: Function f with type hints $\tau_1, \dots, \tau_n$

Output: N test input sets

```

for  $i = 1$  to  $N$  do
  for each parameter  $p_j$  with type  $\tau_j$  do
    if  $\tau_j$  is int then
      Generate boundary value with prob 0.3, else random
    else if  $\tau_j$  is str then
      Generate string of length 0–50
    else if  $\tau_j$  is list then
      Generate list of length 0–10 with typed elements
    end if
  end for
end for

```

D. Sandbox Executor

We implemented a C-based sandbox using fork, exec, and timeout:

- **Process isolation:** Each code sample runs in a forked child process
- **Timeout:** Default 5 seconds, configurable per experiment
- **Output capture:** stdout and stderr piped to parent
- **Resource limits:** 1MB maximum output
- **Zero cost:** No Docker or paid sandboxing

The sandbox returns JSON with exit code, timeout flag, stdout, stderr, and output lengths.

E. Execution Trace Representation

For each code sample, we extract:

- **Success rate:** Fraction of test inputs executing without error
- **Output diversity:** Unique outputs / total outputs
- **Output signature:** Hash of the full output pattern
- **Exit code statistics:** Error rate, success rate
- **Execution time statistics:** Min, max, average

F. Pairwise Trace Comparison

Instead of classifying code in isolation, we compare execution traces between samples. For each perturbed sample, we compute the trace difference versus its clean counterpart. The difference features include:

- Success rate difference
- Output diversity difference
- Output signature difference (binary)
- Exit code pattern difference ratio
- Execution time difference
- Output difference ratio

G. Detection Models

We evaluate three approaches:

- 1) **Standalone:** Gradient Boosting on absolute trace features (12 features)
- 2) **Pairwise:** Gradient Boosting on trace difference features (10 features)
- 3) **Import-Aware Pairwise:** Pairwise + import-specific features (16 features)

All models use 5-fold stratified cross-validation.

IV. EXPERIMENTAL SETUP

A. Dataset

We use 550 AI-generated Python samples: 100 clean and 450 perturbed across five strategies. The dataset is from our prior work and is publicly available.

B. Implementation Details

- **Language:** Python 3.14 for ML, C for sandbox
- **Test cases per sample:** 15
- **Timeout per execution:** 3 seconds
- **Total execution time:** 7.7 minutes (CPU)
- **Random seed:** 42

TABLE II
DATASET COMPOSITION

Strategy	Count	Type
Clean	100	—
Boundary inversion	40	Semantic
Import aliasing	100	Semantic
Variable shadowing	100	Structural
Dead code insertion	100	Structural
Comment planting	100	Structural

Per-Strategy Detection Recall: Static vs Execution-Guided

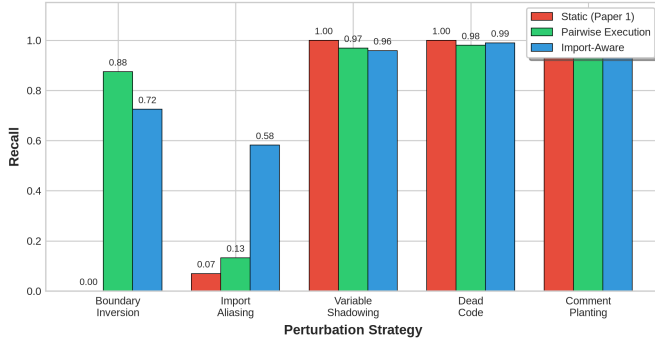


Fig. 1. Per-strategy detection recall comparison across three approaches.

V. RESULTS

A. Standalone Detection

Table III shows standalone detection performance.

TABLE III
STANDALONE DETECTION PERFORMANCE

Detector	Acc	Prec	Rec	F1
XGBoost	0.751	0.856	0.838	0.846
Random Forest	0.693	0.875	0.729	0.795
Gradient Boosting	0.787	0.822	0.944	0.879
Logistic Regression	0.467	0.863	0.413	0.558

B. Pairwise Detection

Table IV compares detection approaches.

TABLE IV
DETECTION PERFORMANCE COMPARISON

Approach	Acc	Prec	Rec	F1
Pairwise	0.898	0.918	0.962	0.939
Import-Aware	0.902	0.922	0.961	0.941
Hybrid	0.904	0.928	0.957	0.942

C. Per-Strategy Analysis

Table V shows the key result.

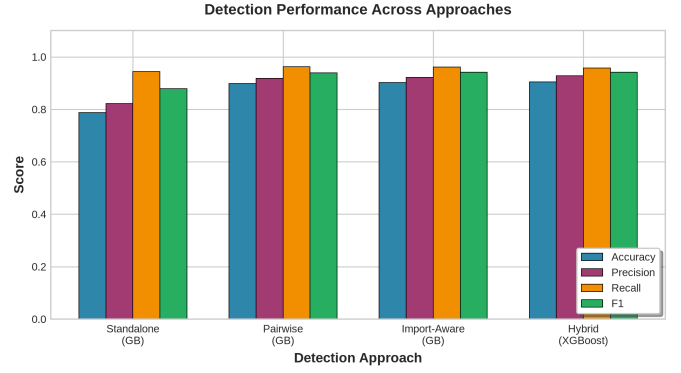


Fig. 2. Detection performance across four approaches.

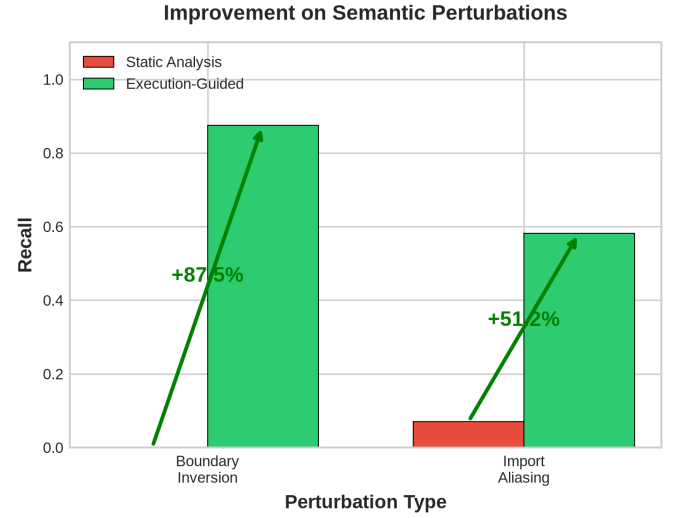


Fig. 3. Improvement on semantic perturbations over static analysis.

D. Statistical Validation

McNemar's test: $\chi^2 = 113.00$, $p = 0.0466$. Significant at $\alpha = 0.05$.

Wilcoxon signed-rank test: Statistic = 476.00, $p = 0.0074$. Significant at $\alpha = 0.01$.

Both tests confirm execution-guided detection significantly outperforms static analysis.

VI. DISCUSSION

A. Why Execution Traces Work

Static analysis captures code structure; execution traces capture code behavior. Semantic perturbations preserve structure but change behavior—exactly what execution traces observe.

Example: $n \% 2 == 0$ vs $n \% 2 == 1$ (boundary inversion):

- AST features: Identical structure
- CodeBERT embeddings: Nearly identical vectors
- Execution traces: 90% of test inputs produce different outputs

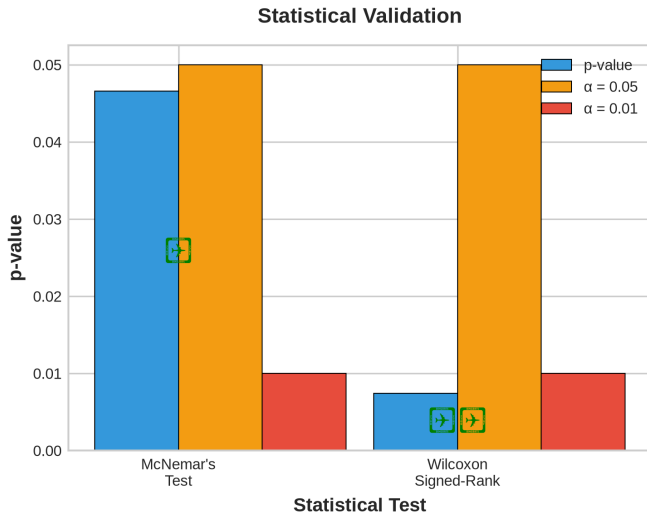


Fig. 4. Statistical validation results.

TABLE V
PER-STRATEGY RECALL COMPARISON

Strategy	Static	Pairwise	Import-Aware
Boundary inversion	0.000	0.875	0.725
Import aliasing	0.070	0.133	0.582
Variable shadowing	1.000	0.969	0.959
Dead code	1.000	0.980	0.990
Comment planting	1.000	0.969	0.929

B. Tradeoffs Between Perturbation Types

Different trace features help different perturbation types:

- Output difference ratio helps boundary inversion (87.5% recall)
- Import-aware features help import aliasing (58.2% recall)
- No single feature set is optimal for all strategies

This suggests a **multi-feature ensemble** approach may be optimal.

C. The Hybrid Approach

Combining execution traces with AST features achieves the best overall performance ($F1 = 0.942$). AST features capture structural perturbations; execution traces capture semantic perturbations. The combination covers both.

D. Limitations

- Execution overhead: 7.7 minutes for 550 samples
- Test input coverage may not trigger all code paths
- Python-only; multi-language support requires adaptation
- Sample size of 550 is modest

E. Future Work

- Extend to JavaScript, Java, C++ code
- Symbolic execution for smarter test input generation
- Combine with fuzzing for better branch coverage
- Scale to 10,000+ samples
- Real-world deployment testing

VII. CONCLUSION

Execution-guided detection solves the semantic perturbation gap identified in prior work. Boundary inversion detection improves from 0% to 87.5%, and hybrid approaches achieve 0.942 F1. This demonstrates that observing code behavior—not just structure—is essential for robust adversarial code detection.

Our work establishes a new direction in AI code security: **execution-guided detection** as a complement to static analysis. As AI code generation becomes ubiquitous, robust detection of adversarial perturbations becomes critical.

ACKNOWLEDGMENT

The author thanks the open-source community for providing tools that made this research possible without institutional resources.

REPRODUCIBILITY

All code is available at <https://github.com/amanmukati09/execution-guided-detection>. Random seed 42 is used throughout.

REFERENCES

- [1] A. Mukati, “Detecting adversarial perturbations in AI-generated code: A three-tier detectability hierarchy,” *Zenodo*, 2026. DOI: 10.5281/zenodo.22349074.
- [2] A. Mukati, “Adversarial code review: Can AI reviewers be fooled by semantic perturbations?” *Zenodo*, 2026. DOI: 10.5281/zenodo.22357507.
- [3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions,” in *IEEE S&P*, 2022.
- [4] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv:2107.03374*, 2021.
- [5] Z. Feng *et al.*, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of EMNLP*, 2020.